

Unit 03

SQL QUERIES

Prof. Ashwini Kumar Mathur
[School of Computer Science and Engineering]

Outlines

1. Introduction to SQL
2. Characteristics of SQL, advantages of SQL.
3. SQL data type and literals,
4. Types of SQL commands,
5. SQL operators and their procedure,
6. Tables, views and indexes,
7. Queries and subqueries,
8. Aggregate functions. [Refer to Lab Experiment 03].
9. Insert, update and delete operations, [Refer to Lab Experiment 02] .
10. Joins, Unions, Intersection, Set Difference, Triggers and Procedures in SQL.

[Introduction to SQL] What is SQL?

Structured query language (SQL) is a Declarative query language for storing and processing information in a **relational database**.

A **relational database** stores information in tabular form, with rows and columns representing different data attributes and the various relationships between the data values.

You can use **SQL statements to store, update, remove, search, and retrieve information from the database**. You can also use SQL to maintain and optimize database performance.

History of SQL

SQL was invented in the 1970s based on the relational data model. It was initially known as the structured English query language (**SEQUEL**). The term was later shortened to SQL.

Oracle, formerly known as Relational Software, became the first vendor to offer a commercial SQL relational database management system.

What are the components of a SQL system?

Relational database management systems use **structured query language (SQL) to store and manage data**. The system stores multiple database tables that relate to each other.

Examples

MS SQL Server, MySQL, or MS Access are examples of relational database management systems.

The following are the components of such a system : SQL Tables, SQL Statements and Stored Procedures

Characteristics of SQL

1. **Declarative Language** – SQL focuses on **what data is needed** rather than *how* to retrieve it.
2. **Standardized Language** – It follows **ANSI and ISO** standards for database management.
3. **Portable** – Can be used across different database systems like MySQL, PostgreSQL, SQL Server, and Oracle.
4. **User-Friendly** – SQL is **easier to learn** compared to procedural languages.
5. **Efficient Data Retrieval** – Provides quick and optimized data retrieval using indexing and optimization techniques.
6. **Scalability** – Suitable for both **small and large-scale** databases.
7. **Security and Access Control** – Supports role-based permissions, encryption, and authentication.

Features of SQL

SQL provides various functionalities to manage data effectively:

A. Data Querying & Manipulation

- ✓ **SELECT** – Retrieves data from a database.
- ✓ **INSERT, UPDATE, DELETE** – Modifies data within tables.
- ✓ **ORDER BY, GROUP BY, HAVING** – Sorts and groups data.

B. Database Structure Definition

- ✓ **CREATE, ALTER, DROP** – Used to define and modify database schema.
- ✓ **Constraints** – Enforces data integrity using PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, and NOT NULL.

C. Transaction Control

- ✓ **COMMIT** – Saves all changes made in a transaction.
- ✓ **ROLLBACK** – Reverts uncommitted transactions.
- ✓ **SAVEPOINT** – Sets a save point within a transaction for partial rollback.

D. Joins and Relationships

- ✓ **INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN** – Combines data from multiple tables.
- ✓ **SELF JOIN** – Joins a table with itself.

E. Advanced Functions

- ✓ **Aggregate Functions** – COUNT, SUM, AVG, MIN, MAX.
- ✓ **String Functions** – CONCAT, LENGTH, LOWER, UPPER.
- ✓ **Mathematical Functions** – ROUND, CEIL, FLOOR, ABS.
- ✓ **Date Functions** – NOW(), CURDATE(), DATEDIFF().

F. Views and Indexing

- ✓ **VIEWS** – Provides virtual tables for simplified querying.
- ✓ **INDEXES** – Enhances search performance.

MySQL Data-types and Literals

1. Numeric Data Types
2. Date and Time Data Types
3. String Data Types
4. Spatial Data Types
5. The JSON Data Type

Numeric Data Types

Used for storing numbers, including integers and decimals.

SQL Data Type	Description	Typical Range/Precision	Storage Size	Notes
Integer Types				
TINYINT	Very small integer	-128 to 127 (signed), 0 to 255 (unsigned)	1 byte	Often used for boolean flags (0 or 1) or small counts.
SMALLINT	Small integer	-32,768 to 32,767	2 bytes	Suitable for moderate-sized whole numbers.
INT / INTEGER	Integer	-2,147,483,648 to 2,147,483,647	4 bytes	The most common integer type for general-purpose use.
BIGINT	Large integer	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	8 bytes	Used for very large whole numbers, such as IDs.
Floating-Point Types				
FLOAT	Single-precision floating-point	Varies (approx. 6 decimal digits precision)	4 bytes	May be called REAL in some systems.
DOUBLE / DOUBLE PRECISION	Double-precision floating-point	Varies (approx. 15 decimal digits precision)	8 bytes	Offers higher precision than FLOAT.
REAL	Floating point number	varies similar to float.	4 bytes	May be a synonym for float, or have slight variations depending on the system.
Decimal Types				
DECIMAL / NUMERIC	Exact numeric with user-defined precision and scale	Varies depending on precision and scale	Variable	Often used for financial data where exact precision is required. Syntax is usually DECIMAL(p, s) or NUMERIC(p, s), where 'p' is precision (total digits) and 's' is scale (decimal places).

Date and Time Data Types

Used for handling dates, times, and timestamps.

Data Type	Storage (bytes)	Date Range	Accuracy
DATETIME	8	January 1, 1753 to December 31, 9999	3-1/3 milliseconds
SMALLDATETIME	4	January 1, 1900 to June 6, 2079	1 minute
DATETIME2	6 to 8	January 1, 0001 to December 31, 9999	100 nanoseconds
DATE	3	January 1, 0001 to December 31, 9999	1 day
TIME	3 to 5		100 nanoseconds
DATETIMEOFFSET	8 to 10	January 1, 0001 to December 31, 9999	100 nanoseconds

String Data Types

Used for storing text and character-based data.

Data Type	Description	Example	Default Value
<code>CHAR(n)</code>	Fixed-length string (1 to 255)	'ABC' (CHAR(3))	NULL
<code>VARCHAR(n)</code>	Variable-length string (1 to 65535)	'Hello World'	NULL
<code>TEXT</code>	Large text data (up to 4GB)	'This is a long text...'	NULL
<code>BLOB</code>	Binary large object for storing files	(Binary Data)	NULL

Spatial Data Types

Used for geographical and spatial data representation.

Data Type	Description	Example
GEOMETRY	Stores any geometric shape	Point, Line, Polygon
POINT	Stores a single point	(X, Y) Coordinates
LINESTRING	Stores a series of points forming a line	Line(X1 Y1, X2 Y2)
POLYGON	Stores an area defined by points	Polygon(X1 Y1, X2 Y2, X3 Y3)

The JSON Data Type

Used for storing JSON (JavaScript Object Notation) data in structured form.

Data Type	Description	Example
JSON	Stores JSON-formatted data	<code>{"name": "Alice", "age": 30}</code>

Types of **SQL** Commands

Data Definition Language (DDL)

Defines the structure of the database.

CREATE – Creates a new database object (table, view, index, etc.).

ALTER – Modifies an existing database object.

DROP – Deletes a database object.

TRUNCATE – Removes all records from a table but keeps its structure.

COMMENT – Adds comments to the database schema.

RENAME – Renames a database object.

Types of **SQL** Commands (Cont ..)

Data Manipulation Language (DML)

Manipulates data within tables.

INSERT – Adds new records.

UPDATE – Modifies existing records.

DELETE – Removes records from a table.

MERGE – Combines INSERT, UPDATE, and DELETE in a single statement.

Data Query Language (DQL)

Retrieves data from the database.

SELECT – Fetches data from tables.

Types of **SQL** Commands (Cont ..)

Data Control Language (DCL)

Manages permissions and access control.

GRANT – Provides access privileges to users.

REVOKE – Removes access privileges from users.

Types of **SQL** Commands (Cont ..)

Transaction Control Language (TCL)

Controls transactions within the database.

COMMIT – Saves all changes made in the current transaction.

ROLLBACK – Undoes changes since the last COMMIT.

SAVEPOINT – Sets a savepoint within a transaction to allow partial rollbacks.

SET TRANSACTION – Defines properties for a transaction.

SQL Operators and their Usages

Arithmetic Operators

Used for mathematical operations on numerical data.

Operator	Description	Example
+	Addition	<code>SELECT 10 + 5;</code> → Output: 15
-	Subtraction	<code>SELECT 10 - 5;</code> → Output: 5
*	Multiplication	<code>SELECT 10 * 5;</code> → Output: 50
/	Division	<code>SELECT 10 / 2;</code> → Output: 5
%	Modulus (Remainder)	<code>SELECT 10 % 3;</code> → Output: 1

SQL Operators and their Usages (Cont..)

Comparison Operators

Used for comparing values in conditions (used in WHERE, HAVING, etc.).

Operator	Description	Example
=	Equal to	<code>SELECT * FROM Employees WHERE Age = 30;</code>
!= or <>	Not equal to	<code>SELECT * FROM Employees WHERE Age <> 30;</code>
>	Greater than	<code>SELECT * FROM Employees WHERE Salary > 50000;</code>
<	Less than	<code>SELECT * FROM Employees WHERE Age < 25;</code>
>=	Greater than or equal to	<code>SELECT * FROM Employees WHERE Age >= 30;</code>
<=	Less than or equal to	<code>SELECT * FROM Employees WHERE Salary <= 50000;</code>

SQL Operators and their Usages (Cont..)

Logical Operators

Used for combining multiple conditions.

Operator	Description	Example
AND	Returns true if both conditions are met	<code>SELECT * FROM Employees WHERE Age > 25 AND Salary > 50000;</code>
OR	Returns true if at least one condition is met	<code>SELECT * FROM Employees WHERE Age < 25 OR Salary > 50000;</code>
NOT	Negates a condition	<code>SELECT * FROM Employees WHERE NOT Age = 30;</code>

SQL Operators and their Usages (Cont..)

Bitwise Operators

Used to perform bit-level operations.

Operator	Description	Example
<code>&</code>	Bitwise AND	<code>SELECT 5 & 3;</code> → Output: 1
<code> </code>	Bitwise OR	
<code>^</code>	Bitwise XOR	<code>SELECT 5 ^ 3;</code> → Output: 6
<code>~</code>	Bitwise NOT	<code>SELECT ~5;</code> → Output: -6

SQL Operators and their Usages (Cont..)

Special Operators

Used for pattern matching and checking NULL values.

Operator	Description	Example
IN	Matches any value in a list	<code>SELECT * FROM Employees WHERE DepartmentID IN (1, 2, 3);</code>
NOT IN	Excludes values in a list	<code>SELECT * FROM Employees WHERE DepartmentID NOT IN (1, 2, 3);</code>
BETWEEN	Checks if a value is within a range	<code>SELECT * FROM Employees WHERE Age BETWEEN 25 AND 35;</code>
LIKE	Matches a pattern	<code>SELECT * FROM Employees WHERE Name LIKE 'A%';</code>
IS NULL	Checks for NULL values	<code>SELECT * FROM Employees WHERE Address IS NULL;</code>
IS NOT NULL	Checks for non-NULL values	<code>SELECT * FROM Employees WHERE Address IS NOT NULL;</code>

What is Table ?

1. A table is a **physical** structure that stores data permanently.
2. It contains columns and rows.
3. You can insert, update, and delete data directly in a table.

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Age INT,  
    Salary DECIMAL(10,2),  
    DepartmentID INT,  
    HireDate DATE,  
    Address VARCHAR(255)  
);
```

What is View in MySQL

A **View** in MySQL is a **virtual table** based on the result of a SQL query. It does not store data physically but displays data dynamically from one or more tables.

Features of Views:

1. Acts as a virtual table
2. Simplifies complex queries
3. Enhances security by restricting access to certain columns
4. Improves data abstraction
5. Can be used to present a subset of data

Example of **view**:

Employee Table:

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Age INT,  
    Salary DECIMAL(10,2),  
    DepartmentID INT,  
    HireDate DATE,  
    Address VARCHAR(255)  
);
```

Employee01 View:

```
CREATE VIEW Employee01 AS  
SELECT EmployeeID, Name, Salary  
FROM Employees  
WHERE Salary > 50000;
```

Difference between **Table** and **View**

Feature	Table	View
Stores Data	✓ Yes	✗ No (Only fetches data from a table)
Physical or Virtual?	Physical	Virtual
Can Insert/Update/Delete?	✓ Yes	⚠ Limited (Depends on complexity)
Based on Other Tables?	✗ No	✓ Yes
Performance	Faster for inserts/updates	Slower if complex

Indexes in MySQL

An **Index** in MySQL is a data structure that improves the speed of queries on a table.

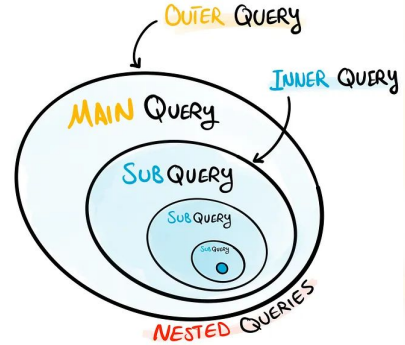
Indexes are used to locate data faster without scanning the entire table.

Creating the index

```
CREATE INDEX idx_name ON Employees(Name);
```

Note: This creates an index on the "Name" column, making searches faster.

Queries and Subqueries



A **query** is an operation that retrieves data from one or more tables or views.

In this reference, a top-level SELECT statement is called a **query**, and a query nested within another SQL statement is called a **subquery**.

Example:

Table: Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

```
SELECT *  
FROM Customers  
WHERE age = (  
    SELECT MIN(age)  
    FROM Customers  
);
```

----- Nested Query

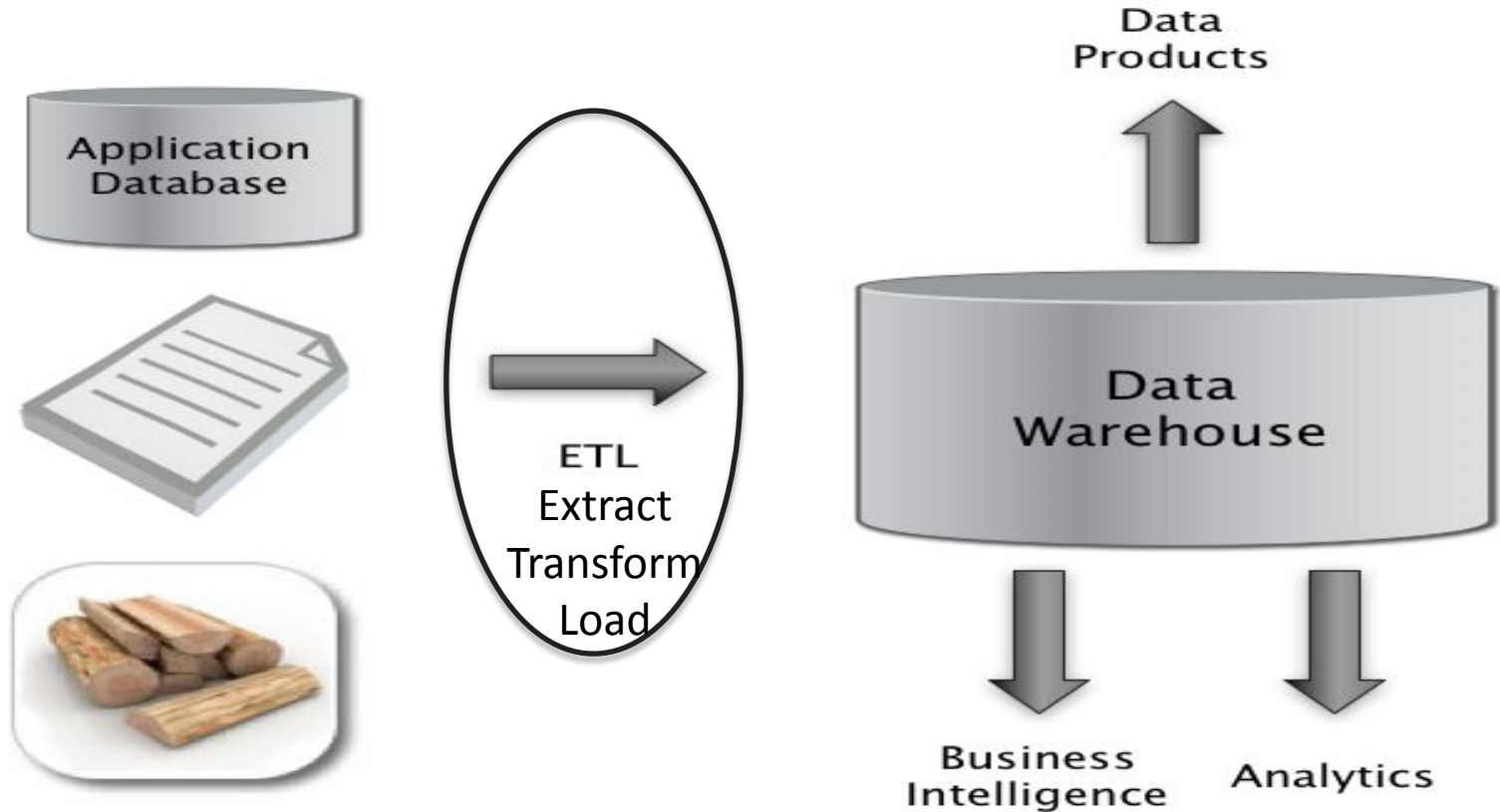
customer_id	first_name	last_name	age	country
2	Robert	Luna	22	USA
3	David	Robinson	22	UK

Combining [JOINS] **Multiple Tables**

SQL intermediate level Queries

Notes : [Click here to open the notes](#)

The **Big Picture**



Joining Multiple Tables

❑ A Join

- Combines data from multiple tables using foreign key references

❑ Syntax

```
SELECT column1, column2, ...
FROM table1, table2
WHERE table1.joincolumn = table2.joincolumn
AND search_condition(s);
```

```
SELECT s_id, s_last, f_last
FROM student, faculty
WHERE student.f_id = faculty.f_id
AND f_last IN ('Marx', 'Zhulin');
```

Joining Multiple Tables (continued)

- ❑ Must qualify column name in SELECT clause
 - Specify name of table that contains column followed by period then column name
 - Example: `SELECT s_id, s_last, student.f_id`
- ❑ Join condition
 - Specifies table names to be joined and column names on which to join tables
 - Example: `WHERE student.f_id = faculty.f_id`

STUDENT

S_ID	S_LAST	S_FIRST	F_ID
JO100	Jones	Tammy	2
PE100	Perez	Jorge	3
MA100	Marsh	John	2
SM100	Smith	Mike	1
JO101	Johnson	Lisa	2
NG100	Nguyen	Ni	4

FACULTY

F_ID	F_LAST
1	Marx
2	Zhulin
3	Langley
4	Brown
5	Sealy

Shared key values

Figure 3-42 Joining two tables based on shared key values

Inner Joins

- ❑ Simplest type of join
- ❑ Also called: Theta join, Equi-join, Natural join
- ❑ VALUES in one table **equal** to values in other table
- ❑ Query design diagram helps get the query right

STUDENT

S_ID	S_LAST	S_FIRST	F_ID
JO100	Jones	Tammy	2
PE100	Perez	Jorge	3
MA100	Marsh	John	2
SM100	Smith	Mike	1
JO101	Johnson	Lisa	2
NG100	Nguyen	Ni	4

```
SELECT s_id, s_last, s_first, student.f_id,  
f_last  
FROM student, faculty  
WHERE student.f_id = faculty.f_id;
```

FACULTY

F_ID	F_LAST
1	Marx
2	Zhulin
3	Langley
4	Brown
5	Sealy

Shared key values

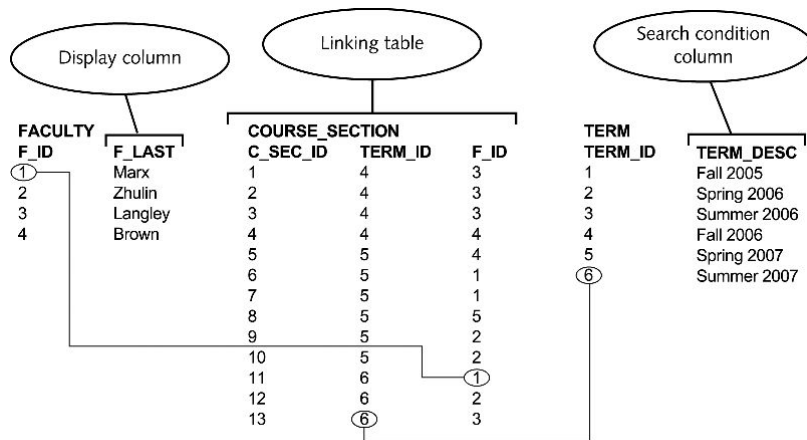
Could be replaced by:

```
FROM Student NATURAL JOIN faculty;
```

Figure 3-42 Joining two tables based on shared key values

Display column, search column, join column

- ❑ **Display columns:** appear in SELECT clause
- ❑ **Search columns:** appear in search condition
- ❑ **Join columns:** primary key and foreign key column on which you join the tables.
- ❑ **Linkage table:** contains join column to link other tables through foreign key values.



```
SELECT f_last
FROM faculty, course_section, term
WHERE faculty.f_id = course_section.f_id
AND course_section.term_id = term.term_id
AND term_desc = 'Summer 2007';
```

Figure 3-44 Joining three tables

Deriving a SQL Query From a Query Design Diagram

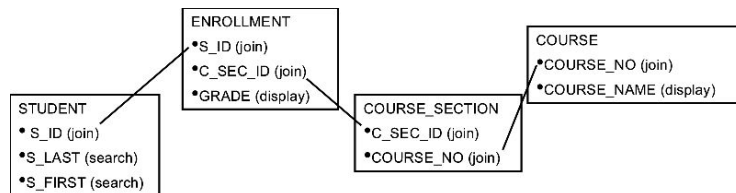


Figure 3-46 Query design diagram

Step	Process	Result
1	Create the SELECT clause by listing the display fields	<code>SELECT course_no, grade</code>
2	Create the FROM clause by listing the table names	<code>FROM student, enrollment, course_section, course</code>
3	Create a join condition for every link between the tables	<code>WHERE student.s_id = enrollment.s_id AND enrollment.c_sec_id = course_section.c_sec_id AND course_section.course_no = course.course_no</code>
4	Add additional search conditions for remaining search fields	<code>AND s_last = 'Jones' AND s_first = 'Tammy'</code>

Table 3-11 Deriving a SQL query from a query design diagram

Joins and Inference

- Chaining relations together is the basic inference method in relational DBs. It produces new relations (effectively new facts) from the data:

```
SELECT S.name,  
M.mortality  
FROM Students S,  
Mortality M
```

Name	Race
Socrates	Man
Thor	God
Barney	Dinosaur
Blarney stone	Stone

Race	Mortality
Man	Mortal
God	Immortal
Dinosaur	Mortal
Stone	Non-living

Joins and Inference

- Chaining relations together is the basic inference method in relational DBs. It produces new relations (effectively new facts) from the data:

**SELECT S.name,
M.mortality
FROM Students S,
Mortality M
WHERE**

Name	Mortality
Socrates	Mortal
Thor	Immortal
Barney	Mortal
Blarney stone	Non-living

E

TYPES OF SQL JOIN OPERATIONS

INNER JOIN

OUTER JOIN

- LEFT OUTER JOIN
- RIGHT OUTER JOIN
- FULL OUTER JOIN

CROSS JOIN

THETA JOIN , EQUI JOIN AND NATURAL JOIN

JOIN TABLE OVERVIEW

(INNER) JOIN: Returns records that have matching values in both tables

LEFT (OUTER) JOIN: Returns all records from the left table, and the matched records from the right table

RIGHT (OUTER) JOIN: Returns all records from the right table, and the matched records from the left table

FULL (OUTER) JOIN: Returns all records when there is a match in either left or right table

SQL Inner Joins

```
SELECT S.name, E.classid  
FROM Students S (INNER) JOIN Enrolled E  
ON S.sid=E.sid
```

S

S.name	S.sid
Jones	11111
Smith	22222
Brown	33333

S.name	E.classid
Jones	History105
Jones	DataScience194
Smith	French150

E

E.sid	E.classid
11111	History105
11111	DataScience194
22222	French150
44444	English10

Note the previous version of this query (with no join keyword) is an “Implicit join”

ANOTHER EXAMPLE:

Select matching records on the basis of common roll_number field

roll_number	name
1	Meenakshi
2	Manjili
3	Sheetal
4	Apoorva
5	Keeto
6	Nitin
7	Soumya

roll_number	marks
1	89
3	74
4	100
6	29

INNER JOIN

name	marks
Meenakshi	89
Sheetal	74
Apoorva	100
Nitin	29

SQL Inner Joins

SELECT S.name, E.classid
FROM Students S (INNER) JOIN Enrolled E
ON S.sid=E.sid

S

S.name	S.sid
Jones	11111
Smith	22222
Brown	33333

S.name	E.classid
Jones	History105
Jones	DataScience194
Smith	French150

E

E.sid	E.classid
11111	History105
11111	DataScience194
22222	French150
44444	English10

Unmatched keys

What kind of Join is this?

SELECT S.name, E.classid

FROM ~~Students S~~ ?? ~~Enrolled E~~

ON S.sid=E.sid

S

S.name	S.sid
Jones	11111
Smith	22222
Brown	33333

S.name	E.classid
Jones	History105
Jones	DataScience194
Smith	French150
Brown	NULL

E

E.sid	E.classid
11111	History105
11111	DataScience194
22222	French150
44444	English10

SQL Joins

SELECT S.name, E.classid

FROM Students S LEFT OUTER JOIN

Enrolled E

S

S.name	S.sid
Jones	11111
Smith	22222
Brown	33333

S.name	E.classid
Jones	History105
Jones	DataScience194
Smith	French150
Brown	NULL

E

E.sid	E.classid
11111	History105
11111	DataScience194
22222	French150
44444	English10

ANOTHER EXAMPLE:

Select matching records on the basis of common roll_number field
between the tables and all the records from the left table

roll_number	name
1	Meenakshi
2	Manjili
3	Sheetal
4	Apoorva
5	Keeto
6	Nitin
7	Soumya

roll_number	city
1	Ajmer
3	Bangalore
5	Udaipur
6	Chittor
10	Raipur

LEFT JOIN

name	city
Meenakshi	Ajmer
Manjili	NULL
Sheetal	Bangalore
Apoorva	NULL
Keeto	Udaipur
Nitin	Chittor
Soumya	NULL

What kind of Join is this?

SELECT S.name, E.classid

FROM ~~Students S~~ ?? ~~Enrolled E~~

ON S.sid=E.sid

S

S.name	S.sid
Jones	11111
Smith	22222
Brown	33333

S.name	E.classid
Jones	History105
Jones	DataScience194
Smith	French150
NULL	English10

E

E.sid	E.classid
11111	History105
11111	DataScience194
22222	French150
44444	English10

SQL Joins

SELECT S.name, E.classid

**FROM Students S RIGHT OUTER JOIN
Enrolled E**

S

S.name	S.sid
Jones	11111
Smith	22222
Brown	33333

S.name	E.classid
Jones	History105
Jones	DataScience194
Smith	French150
NULL	English10

E

E.sid	E.classid
11111	History105
11111	DataScience194
22222	French150
44444	English10

Select matching records on the basis of common roll_number field
between the tables and all the records from the right table

roll_number	name
1	Meenakshi
2	Manjili
3	Sheetal
4	Apoorva
5	Keeto
6	Nitin
7	Soumya



roll_number	city
1	Ajmer
3	Bangalore
5	Udaipur
6	Chittor
10	Raipur

RIGHT JOIN

roll_number	city
Meenakshi	Ajmer
Sheetal	Bangalore
Keeto	Udaipur
Nitin	Chittor
NULL	Raipur

SQL Joins

SELECT S.name, E.classid

FROM Students S ? JOIN Enrolled E

ON S.sid=E.sid

S

S.name	S.sid
Jones	11111
Smith	22222
Brown	33333

S.name	E.classid
Jones	History105
Jones	DataScience194
Smith	French150
NULL	English10
Brown	NULL

E

E.sid	E.classid
11111	History105
11111	DataScience194
22222	French150
44444	English10

SQL Joins

SELECT S.name, E.classid

FROM Students S FULL OUTER JOIN Enrolled E

S

S.name	S.sid
Jones	11111
Smith	22222
Brown	33333

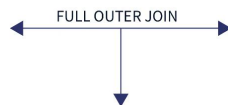
S.name	E.classid
Jones	History105
Jones	DataScience194
Smith	French150
NULL	English10
Brown	NULL

E

E.sid	E.classid
11111	History105
11111	DataScience194
22222	French150
44444	English10

roll_number	name
1	Meenakshi
2	Manjili
3	Sheetal
4	Apoorva
5	Keeto
6	Nitin
7	Soumya

roll_number	city
1	Ajmer
3	Bangalore
5	Udaipur
6	Chittor
10	Raipur



name	city
Meenakshi	Ajmer
Manjili	NULL
Sheetal	Bangalore
Apoorva	NULL
Keeto	Udaipur
Nitin	Chittor
Soumya	NULL
NULL	Raipur

What kind of Join is this?

SELECT S.name, E.classid

FROM ~~Students S~~ ?? ~~Enrolled E~~

ON S.sid=E.sid

S

S.name	S.sid
Jones	11111
Smith	22222
Brown	33333

S.name	E.classid
Jones	History105
Smith	French150

E

E.sid	E.classid
11111	History105
11111	DataScience194
22222	French150
44444	English10

SUMMARY

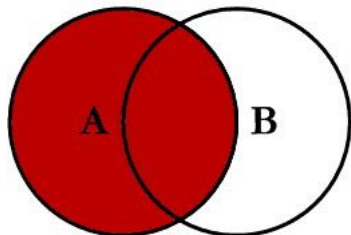
(INNER) JOIN: Returns records that have matching values in both tables

LEFT (OUTER) JOIN: Returns all records from the left table, and the matched records from the right table

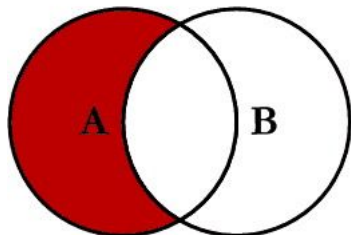
RIGHT (OUTER) JOIN: Returns all records from the right table, and the matched records from the left table

FULL (OUTER) JOIN: Returns all records when there is a match in either left or right table

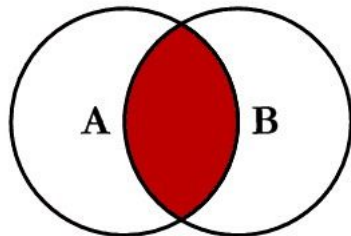
SQL JOINS



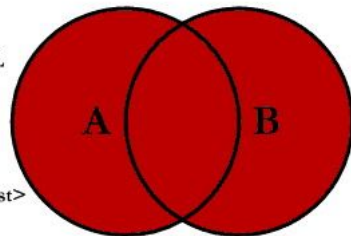
```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



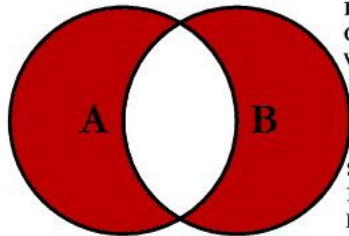
```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```



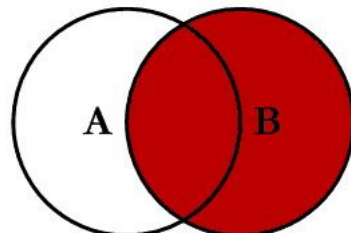
```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



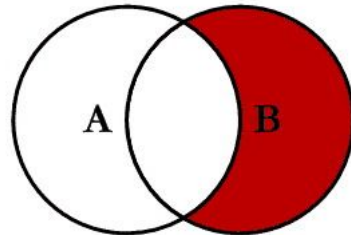
```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```



```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```

What kind of Join is this?

SELECT *

FROM Students S ?? Enrolled E

S

S.name	S.sid
Jones	11111
Smith	22222

E

E.sid	E.classid
11111	History105
11111	DataScience194
22222	French150

S.name	S.sid	E.sid	E.classid
Jones	11111	11111	History105
Jones	11111	11111	DataScience194
Jones	11111	22222	French150
Smith	22222	11111	History105
Smith	22222	11111	DataScience194
Smith	22222	22222	French150

SQL Joins

SELECT *

FROM Students S CROSS JOIN Enrolled E

S

S.name	S.sid
Jones	11111
Smith	22222

E

E.sid	E.classid
11111	History105
11111	DataScience194
22222	French150

S.name	S.sid	E.sid	E.classid
Jones	11111	11111	History105
Jones	11111	11111	DataScience194
Jones	11111	22222	French150
Smith	22222	11111	History105
Smith	22222	11111	DataScience194
Smith	22222	22222	French150

What kind of Join is this?

SELECT *

FROM Students S, Enrolled E

WHERE S.sid = E.sid

S

S.name	S.sid
Jones	11111
Smith	22222

E

E.sid	E.classid
11111	History105
11111	DataScience194
22222	French150

S.name	S.sid	E.sid	E.classid
Jones	11111	11111	History105
Jones	11111	11111	DataScience194
Jones	11111	22222	French150
Smith	22222	22222	French150

Theta Joins

SELECT *

FROM Students S, Enrolled E

WHERE S.sid <= E.sid

S **E**

S.name	S.sid
Jones	11111
Smith	22222

E.sid	E.classid
11111	History105
11111	DataScience194
22222	French150

S.name	S.sid	E.sid	E.classid
Jones	11111	11111	History105
Jones	11111	11111	DataScience194
Jones	11111	22222	French150
Smith	22222	22222	French150

SQL – Join

- A Join is a Query that combines data from multiple tables
 - Multiple tables are specified in the From Clause
 - For two tables to be joined in a sensible manner, they need to have data in common

Example:

Schema: Movies (movie_title, director_id, release_date)
 People(person_fname, person_lname, person_id)

Query: Select movie_title, person_fname, person_lname
 From Movies, People
 Where director_id = person_id

SQL – Joining Condition

- For a useful Join query a joining condition is required
 - Defined in where clause as relationships between columns
 - Multiple conditions may be defined if multiple columns shared
 - More than two tables can be joined in a query

Example: Find people who live in same state as studio

Schema:

Studios(studio_id, studio_state, studio_name, studio_city)

People(person_fname, person_lname, person_id, person_state, person_city)

Query:

Select person_fname, person_lname, studio_name

From Movies, People

Where studio_city = person_city

AND studio_state = person_state

SQL – More than two tables

Example: Get title, director, studio, city for all movies in the database

Schema:

Studios(studio_id, studio_state, studio_name, studio_city)

People(person_fname, person_lname, person_id, person_state, person_city)

Movies(movie_title, director_id, studio_id)

Query:

Select M.movie_title, M.studio_id, P.person_fname, P.person_lname, S.studio_city

From Movies M, People P, Studio S

Where M.director_id = P.person_id

AND M.studio_id = P.person_id

Aggregate Queries

Including reference counts in the lookup tables allows you to perform aggregate queries on those tables alone:

Average age of users, most popular location,...

Tweet id	User id	Location id	Body
11	111	1111	I need a Jamba Juice
22	111	1111	Cal Soccer rules
33	111	2222	Why do we procrastinate?
44	222	3333	Close your eyes and push “go”

U.id	Name	Count	Attr..
111	Jones	3	
222	Smith	1	

L.id	Name	Count	Attr...
1111	Berkeley	2	
2222	Oakland	1	
3333	Hayward	1	

Question 01 .

What is a join in SQL?

- a) A function used to add new data to an existing table
- b) A function used to combine data from two or more tables based on a related column
- c) A function used to group data in a table by a certain criteria

What is a primary key in a database table?

- a) A column that uniquely identifies each row in the table
- b) A column that contains a list of values to be used in a join statement
- c) A column that stores the sum of values in another column

Structured Query Language

DML

More Practices ..

SQL – Select

Select *<List of Columns and expressions (usually involving columns)>*

From *<List of Tables & Join Operators>*

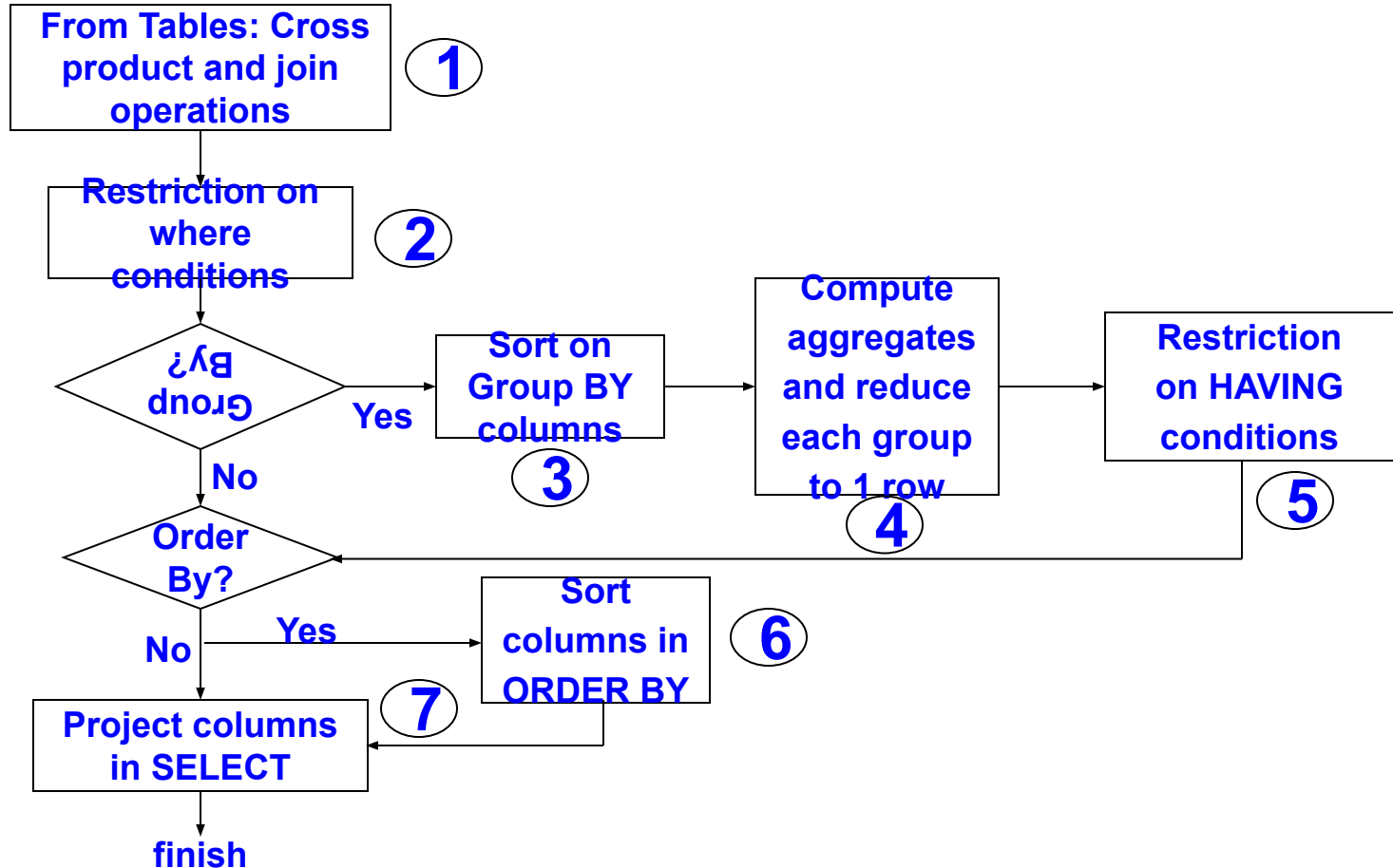
Where *<List of Row conditions joined together by **And**, **Or**, **Not**>*

Group By *<list of grouping columns>*

Having *<list of group conditions connected by **And**, **Or**, **Not**>*

Order By *<list of sorting specifications>*

Conceptual Evaluation



SQL – DISTINCT

- Eliminates all the duplicate entries in the table resulting from the query.

Syntax:

```
Select [DISTINCT] select_list  
From table[, table, ...]  
[Where expression]  
[Order By expression]
```

Example:

```
Select DISTINCT studio_id, director_id  
From Movies
```

<u>studio_id</u>	<u>director_id</u>
1	1
2	2
2	10
3	1
3	9

SQL – Order By

- Used to sort the results based on contents of a column
- Multiple levels of sort can be done by specifying multiple columns
- An expression can be used in Order By clause

Syntax:

Select function(column)

From table1 [, table2 ...]

[Where condition]

[Order By {Column | alias | position} [ASC | DESC]]

SQL – Order By

Example: Sort Movies by profits in Ascending order

Select MovieTitle, Gross, Budget, (Gross – Budget) as profits

From movies

Order BY profits

Movie_title	Gross	Budget	Profit
	67.5	70	-2.5
Great Escape			
Upside Down	54	50	4
Green Warrior	96	80	16
Blue Oranges	28	7	21

Aggregate Queries – Group By

- Categorizes the query results according to the contents of a column in the database
- Multiple levels of subgroups can be created by specifying multiple columns

Syntax:

Select column1, [column2, ...]

From table [, table ...]

[Where condition]

Group By column1, [column2,]

Having [Condition]

Aggregate Queries – Group By

Example: Get # of movies by each director for each studio

Select studio_id, director_id, count(*)

From Movies

Group By director_id, studio_id

Example: Get # of movies by each studio ordered by studio_id

Select studio_id, count(*)

From Movies

Group By studio_id

Order By studio_id

Aggregate Queries – Group By

Example:

```
Select studio_id, Sum(budget)
From movies
Group by studio_id
Having Sum(budget) > 60
```

Example:

```
Select studio_id, count(*)
From Movies
Group By studio_id
Order By studio_id
```


Aggregate Queries

- Aggregate queries provides a more holistic view of the data by further processing the retrieved data.
- They can work on
 - On all the rows in a table
 - A subset of rows in a table selected using a where clause
 - Groups of selected data organized using Group By clause.

Syntax:

Select function(column)

From <list of tables>

Where <condition>

Group By <list of columns>

Having <condition>

Aggregate Queries

- Functions:
 - Sum() Returns a sum of the column
 - Count() Returns a total number of rows returned by a query
 - Avg() Returns the average of a column
 - Min() Returns minimum value of the column returned by query
 - Max() Returns maximum value of the column returned by query

Notes 1: Count function does not include columns containing null values in total

Notes 2: Count can be used with distinct to count the number of distinct rows

Example:

Query: Select sum(budget)
From movies
Where studio_id = 3

Output: Sum(budget)

65.1

Stored Procedure

A stored procedure is a **Pre-compiled SQL code block**, or a **set of SQL statements**, that is stored in a database.

It can be executed by name, offering reusability, performance benefits, and improved maintainability.

Link: [Click here to open the link \[Detailed example \]](#)

Triggers and Stored Procedures

A **trigger** is an event-driven action that executes **automatically** before or after an INSERT, UPDATE, or DELETE operation on a table.

Link:

<https://docs.google.com/document/d/1XT13HWN9FfBvFEpM8jn7wmEkk4z33iDPoeai6zZXbjs/edit?usp=sharing>

A **stored procedure** is a precompiled SQL code block that can be **executed manually** with parameters.

Link:

<https://docs.google.com/document/d/1XT13HWN9FfBvFEpM8jn7wmEkk4z33iDPoeai6zZXbjs/edit?usp=sharing>

Difference between Trigger and Stored Procedure

Feature	Trigger	Stored Procedure
Execution	Automatically	Manually (CALL)
When It Runs	On INSERT, UPDATE, DELETE	When explicitly invoked
Accepts Parameters?	✗ No	✓ Yes
Used For	Logging, validation, audit	Complex business logic

End of Unit 03 : Feel free to ask questions !!

Practice Questions: [Click here to view the Documents](#)